

Class 09: Data Fetching & Connecting to API

2110507 Software Development Practice II

Machigar Ongtang, Ph.D.

Next.js: Fetching Data



- การ Fetch ข้อมูล เป็นหนึ่งในกลไกการทำงานหลักของ Web Application สำหรับ Next.js การ fetch ข้อมูลสามารถกระทำได้ทั้ง
  - การ Fetch ข้อมูลบน Server-Side
  - การ Fetch ข้อมูลบน Client-Side

Data Fetching Patterns

- React และ Next.js รองรับการ fetch ข้อมูลหลากหลายรูปแบบ โดยมีรูปแบบที่แนะนำ ได้แก่
- Fetching Data on the Server
  - หากเป็นไปได้ ควร fetch data บน server เป็นหลัก เนื่องจาก
    - Server มักเป็นช่องทางที่สามารถเข้าถึง resource ต่าง ๆ เช่น database โดยตรง
    - การประมวลผลข้อมูลที่ได้รับตั้งแต่บน server เพิ่มความปลอดภัยให้กับ application โดยป้องกันการนำส่งข้อมูลที่ sensitive เช่น access tokens และ API keys มายัง client
    - การ fetch ข้อมูลและ render ผลลัพธ์ใน environment เดียวกัน ลดการสื่อสารไปมาระหว่าง server และ client และลดงานบน client
    - เอื้อต่อการ Fetch data หลายครั้งในการสื่อสาร round-trip เดียว แทนที่การส่งหลาย request จาก client

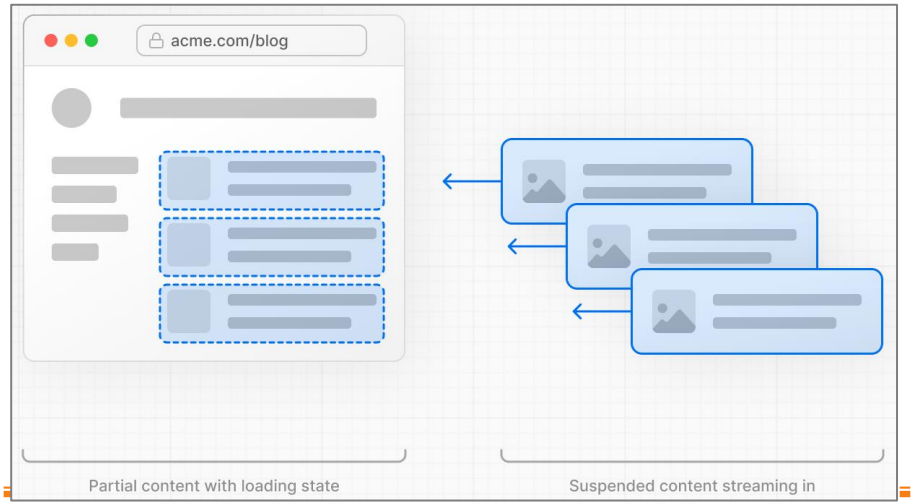
Next.js: Fetching Data



- **Fetching Data on the Server**
  - หากเป็นไปได้ ควร fetch data บน server เป็นหลัก เนื่องจาก
    - ลดการเกิด client-server waterfalls ซึ่งเกิดเมื่อหลาย request ใน route ทำงานขึ้นต่อกันและกัน ซึ่งควรหลีกเลี่ยงให้เกิดเฉพาะเมื่อจำเป็นเท่านั้น เพราะจะทำให้ loading time นาน
    - การ fetch data บน Server มักเป็นการดึงข้อมูลในระยะที่ใกล้กับ data source จึงลดเวลาการทำงานและเพิ่มประสิทธิภาพ
  - Next.js รองรับการ fetch data บน server ผ่านการทำงานบน server component, route handler และ server action
- **Fetching Data Where It's Needed**
  - Fetch data เมื่อจำเป็นต้องใช้ หากผู้ใช้หนึ่ง ๆ ต้องการ data เดียวกันในหลาย component ใน Tree ก็ไม่จำเป็นต้อง fetch data ระดับ Global หรือส่ง data เป็น props ระหว่าง component
  - Fetch request ถูก cache อัตโนมัติ จึงสามารถใช้ `fetch` หรือ `cache` ใน React component ที่ต้องการใช้ data ได้ จากนั้นให้ React บริหารจัดการ โดยไม่ต้องบริหารจัดการประสิทธิภาพเอง จากการ fetch request หลายครั้งสำหรับดึง data เดียวกัน

Next.js: Fetching Data

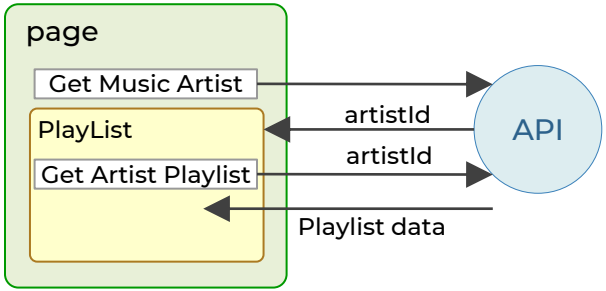
- **Streaming**
  - React มาพร้อม feature การทำ Streaming และ Suspense ที่รองรับการทำ rendering แบบค่อยเป็นค่อยไป โดยทยอย stream ชิ้น UI ที่ได้รับการ render แล้วมายัง client
  - การใช้ Server Components และ nested layouts ทำให้สามารถ render ส่วนต่าง ๆ ของ page ที่ไม่ต้องการข้อมูลได้โดยทันที และแสดง loading state สำหรับส่วนอื่นของ page ที่กำลัง fetch data
    - User ไม่จำเป็นต้องรอหน้า page ทั้งหมดให้ load ก่อนที่จะ interact กับ page



https://nextjs.org/docs/app/building-your-application/data-fetching/patterns

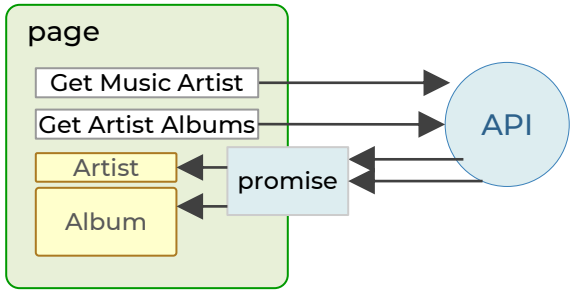
Next.js: Fetching Data

- **Parallel and Sequential Data Fetching**
  - พิจารณา pattern ของการ fetch data ใน React components ทั้งแบบ Parallel (ดึงขนาน) และแบบ Sequential (ดึงอนุกรมต่อกัน)
  - **Sequential Data Fetching** หลาย request ใน route ทำงานขึ้นต่อกันและกัน สร้างให้เกิด waterfalls
    - ใช้เมื่อจำเป็น หากการ fetch ข้อมูล อาศัยผลลัพธ์จากการ fetch ข้อมูลอื่นก่อน
    - ใช้เมื่อจำเป็น หากการ fetch ข้อมูล ต้องรอเงื่อนไขบางอย่างก่อนการ fetch ครั้งต่อไป เพื่อจัดการ resource
    - หลีกเลี่ยง หากเกิดโดยไม่ตั้งใจ จะทำให้ loading time นานเกินความจำเป็น
    - หากมี nested component ที่แต่ละ component มีการ fetch data ของตนเอง (ซึ่งเป็น data request ที่ต่างกัน จึงไม่ได้รับการ cache) จะเกิด **Sequential Data Fetching**
    - สามารถใช้ `loading.js` หรือ `suspense` เพื่อแสดง loading state ในขณะทำการ stream เพื่อหลีกเลี่ยงไม่ให้หน้าใน route นั้นถูก block และบางส่วนของ page ยังโต้ตอบกับผู้ใช้ได้



Next.js: Fetching Data

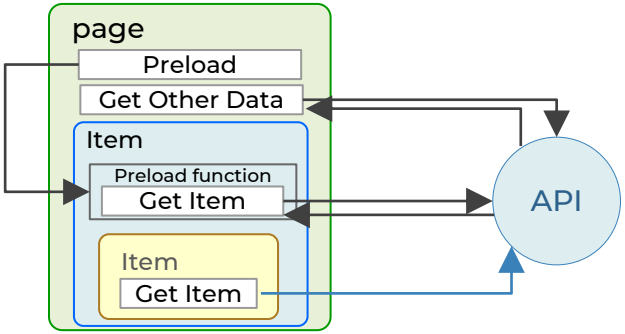
- **Parallel Data Fetching** หลาย request ใน route เริ่ม load data ในเวลาเดียวกัน ลด client-server waterfall และลด loading time โดยรวม
  - เริ่มส่ง request โดยระบุไว้นอก component ที่ใช้ data จากนั้นเรียกใช้จากใน component
  - ลดเวลาโดยส่ง request แบบ parallel แต่ user จะไม่เห็นผลลัพธ์ของการ render จนกว่า promise ทั้งหมดจะถูกดำเนินการเสร็จสิ้น



  - อาจปรับปรุงโดยใช้ **suspense** เพื่อแยกการทำ rendering และแสดงส่วนที่ได้ผลลัพธ์แล้วทันที

Next.js: Fetching Data

- **Preloading Data**
  - ป้องกัน waterfall ผ่านการใช้ preload pattern โดยสร้าง function ที่ทำการ preload เพื่อ optimize parallel data fetching



- นอกจากนั้น ยังสามารถใช้งานหลาย feature ร่วมกันสำหรับ fetch data ได้ เช่น รวมการใช้งาน `cache` ใช้งาน `server-only` feature และใช้ preload pattern ร่วมกัน
  - fetch data, cache response และวางให้การ fetch data เกิดขึ้นบน server เท่านั้น

Asynchronous Operations



การ Fetch Data เป็น Operation ที่ใช้ระยะเวลานาน จึงนิยมใช้การทำงานแบบ Asynchronous

JavaScript promise และ async/await

- ทั้ง `async/await` และ `Promise` เป็นเครื่องมือใน JavaScript สำหรับจัดการ การทำงานแบบ asynchronous
  - เช่น การ request ข้อมูลจาก server และ การอ่านข้อมูลจาก database

- `async/await` เป็น Feature ใน JavaScript ที่ช่วยให้เขียนโค้ดที่ทำงานแบบ asynchronous โดยเขียนในลักษณะเดียวกับการเขียนโค้ดปกติ
  - ทำให้เขียนโค้ดที่ใช้ asynchronous operation ได้ง่ายขึ้น โดยนำ keyword `await` วางไว้หน้าคำสั่ง เพื่อกำหนดให้รอจนกว่าคำสั่งนั้นจะทำงานเสร็จ
  - การใช้งาน `await` ต้องใช้ในฟังก์ชั่นที่ประกาศเป็น `async` เสมอ

```
async function main() {
  await taskOne();
}
main();
```

Asynchronous Operations



- คำสั่ง `fetch()` สำหรับการ Fetch data

```
response = await fetch(url)
```

  - *url* URL endpoint สำหรับการดึงข้อมูล
  - *response* Response Object ที่ได้รับ โดยมี properties ที่สำคัญ เช่น
    - *response*.ok ค่า Boolean ระบุว่า response สำเร็จหรือไม่
    - *response*.status Status Code ของ Response เช่น 200 หมายถึง success
    - *response*.statusText ข้อความแสดง status ของ Response เช่น OK หมายถึง success

    มี method ที่สำคัญ เช่น
    - *response*.json() คืนค่า promise ที่ resolve เป็นผลลัพธ์ของการ parse response body เป็น json
    - *response*.text() คืนค่า promise ที่ resolve เป็นผลลัพธ์ response body ที่เป็น text
  - ตัวอย่าง การใช้งาน `await` ในการทำ network request

```
async function getData() {
    const response = await fetch('https://www.example.com/data.json');
    const data = await response.json();
    return data;
}
```

Asynchronous Operations



- Promise เป็น JavaScript Object ที่เอื้อให้การทำงานแบบ Asynchronous สามารถทำงานแบบรอกันได้
  - โดยปกติการทำงานแบบ Asynchronous เป็นการทำงานแบบไม่รอกัน แต่หลายครั้งที่โปรแกรมต้องการให้การทำงาน ต้องทำอย่างหนึ่งเสร็จก่อนแล้วถึงจะทำอีกอย่างหนึ่ง
  - Promise ใน JavaScript เป็นทางเลือกที่บังคับให้การทำงานสามารถรอกันได้
  - การทำงานของ Promise ให้ผลลัพธ์ได้สองรูปแบบ คือ
    - การทำงานที่สำเร็จ (Resolve หรือ fulfill)
    - การทำงานที่ล้มเหลว (Reject)
  - การสร้าง Promise สร้างด้วยรูปแบบ

```
let promise = new Promise(function(resolve, reject){
    //do something
});
```

    - Constructor ของ Promise มี 1 parameter เป็น callback function รับ argument 2 ตัว
    - กระทำ operation ใน Callback function ถ้าทำงานสำเร็จ ให้เรียก resolve() ถ้าทำงานล้มเหลว ให้เรียก reject()

Asynchronous Operations



- การ Consume Promise ทำได้โดยผูก function เข้าไปกับ promise ด้วยเมธอด .then() และ .catch()

```
let promise = new Promise(function (resolve, reject) {
    const x = "hello";
    const y = "hello"
    if (x === y) {
        resolve();
    } else {
        reject();
    }
});

promise.
    then(function () {
        console.log('Success, Promise is successful');
    }).
    catch(function () {
        console.log('Some error has occurred');
    });
```

ถูกเรียกเมื่อมีการ resolve และได้รับผลลัพธ์

ถูกเรียกเมื่อมีการ reject หรือเมื่อมี error จากการ run

Asynchronous Operations



- **Promise Chaining** เชื่อมหลาย Promise ต่อกัน เพื่อให้ทำงานรอกันแบบเป็นลำดับ
  - เช่น ต้องการให้โปรแกรมทำงานจาก fn1 และต่อด้วย fn2 โดยที่ฟังก์ชันทั้งหมดทำงานแบบ Asynchronous

```
function count(number) {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            console.log(number);
            resolve();
        }, 1000);
    });
}
```

Function return เป็น Promise

```
count('One').then(() => {
    return count('Two');
}).then(() => {
    return count('Three');
}).then(() => {
    return count('Four');
});
```

นำมาต่อกันผ่าน then()

Asynchronous Operations



- Async/Await กับ Promise ใช้งาน Promise ร่วมกับคำสั่ง Async/Await เพื่อใส่การทำงานในรูปแบบ Synchronous โดยโปรแกรมรอจนกว่า Promise จะทำงานเสร็จสิ้น

```
function setTimeoutPromise(delay) {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            resolve();
        }, delay);
    });
}
```

```
async function run() {
    await setTimeoutPromise(500);
    console.log('Task 1 done');

    await setTimeoutPromise(1000);
    console.log('Task 2 done');

    await setTimeoutPromise(2000);
    console.log('All tasks done');
}

run();
```

await ทำงานใน Function ที่เป็น async เท่านั้น

เพิ่ม await หน้า Function ที่ return เป็น Promise เพื่อให้รอการทำงานจนกว่า Promise จะทำเสร็จสิ้น

```
Task 1 done
Task 2 done
All tasks done
```

Source Code: *http://marcuscode.com/lang/javascript/promise*

Asynchronous Operations



- เมธอด `Promise.all()` รับ argument เป็น array (iterable) ของ `Promise` และ return ค่าเป็น `Promise` ที่จะได้รับการ resolve ก็ต่อเมื่อ `Promise` ทั้งหมดที่รับเข้ามาถูก resolve เรียบร้อยทั้งหมด หรือเมื่อ iterable ไม่มี `Promise` เหลือ

```
let resultPromise = Promise.all([promise1, promise2, promise3, …]);
```

Server-Side Data Fetching

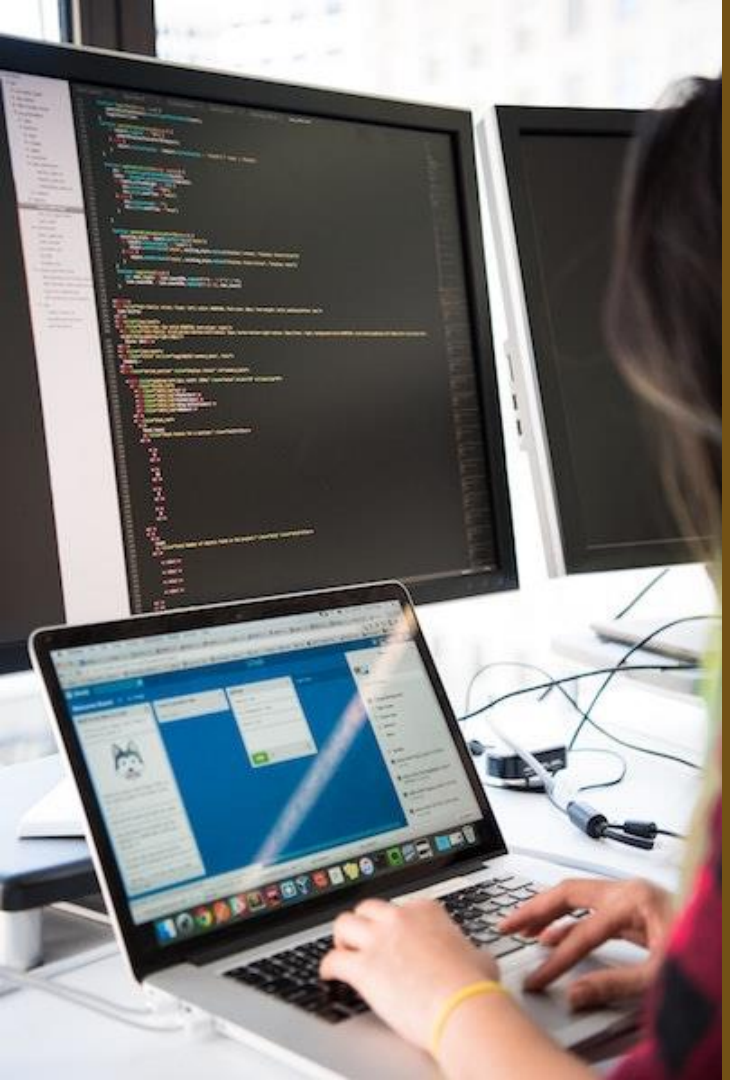


- Next.js ต่อยอด `fetch` Web API เพื่อรองรับการทำ caching และการทำ revalidation สำหรับแต่ละ fetch request บน server
- Cache: React จดจำ fetch request โดยอัตโนมัติในระหว่างการ render React component tree
  - แต่ละ fetch request จะได้รับการ cache ไว้ใน data cache บน Server เพื่อหลีกเลี่ยงการ fetch ข้อมูลที่ไม่จำเป็น โดย data อาจถูก fetch ตอน build-time หรือ request-time จากนั้นจะถูก cache และ reuse ในแต่ละ data request
  - สามารถตั้ง `cache` option สำหรับคำสั่ง `fetch` ได้

```
fetch(`https://...`, { cache: 'option' })
```

option ระบุรูปแบบการ cache

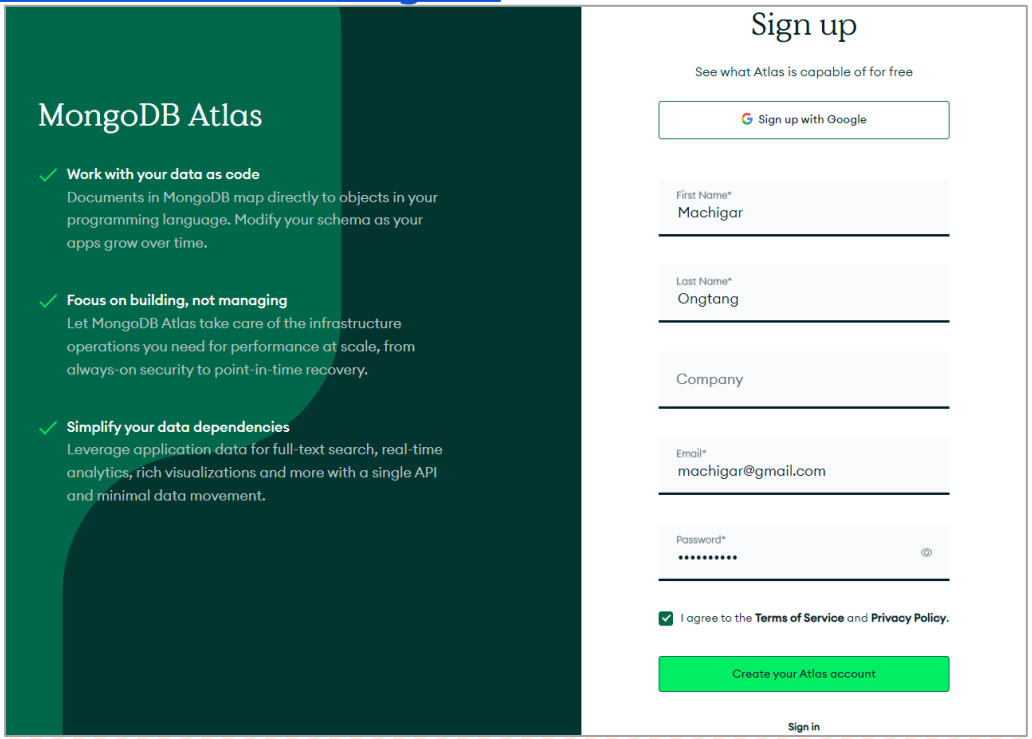
- `force-cache` บังคับการใช้งาน cache (ค่า default)
- `no-store` ไม่ใช้งาน cache โดยดึง data จาก remote server สำหรับทุก request



Workshop 9.1: Setting Up API

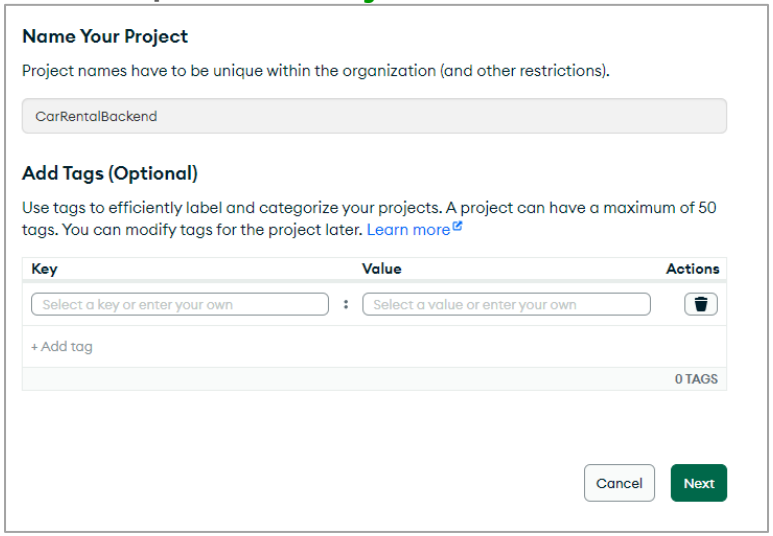
MongoDB Atlas Setup for your project

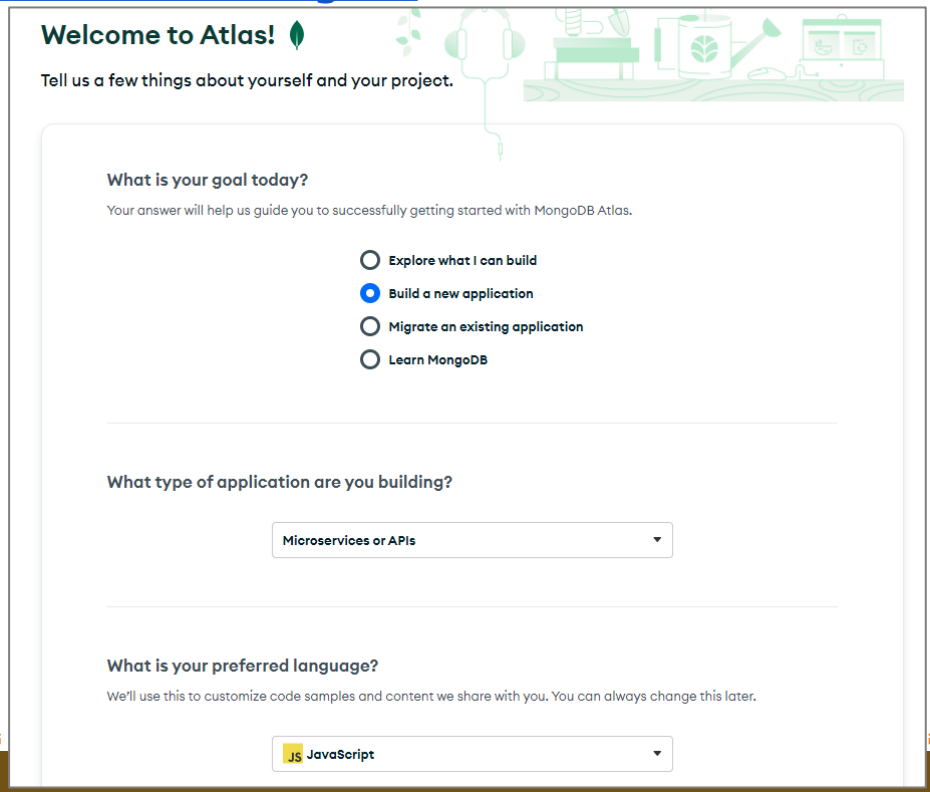
- Visit MongoDB Atlas → Try for Free
  - หรือไปที่ https://www.mongodb.com/cloud/atlas/register
  - Sign-Up with a valid email address
  - Verify your email



MongoDB Atlas Setup for your project

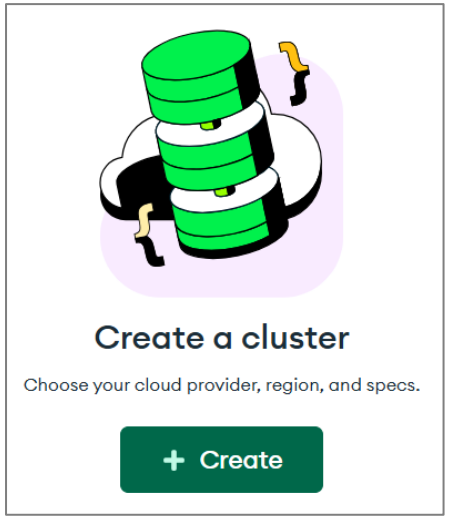
- Visit MongoDB Atlas → Try for Free
  - หรือไปที่ https://www.mongodb.com/cloud/atlas/register
  - Sign-Up with a valid email address
  - Verify your email
  - Setup a New Project





MongoDB Atlas Setup for your project

- Deploy Create a Cluster



Deploy your cluster

Use a template below or set up advanced configuration options. You can also edit these configuration options once the cluster is created.

M10 $0.09/hour
For production applications with sophisticated workload requirements.
STORAGE 10 GB | RAM 2 GB | vCPU 2 vCPUs

Serverless $0.12/1M reads
For application development and testing, or workloads with variable traffic.
STORAGE Up to 1 TB | RAM Auto-scale | vCPU Auto-scale

M0 Free
For learning and exploring MongoDB in a cloud environment.
STORAGE 512 MB | RAM Shared | vCPU Shared

Free forever! Your M0 cluster is ideal for experimenting in a limited sandbox. You can upgrade to a production cluster anytime.

Name
You cannot change the name once the cluster is created.
[redacted]

Automate security setup
Preload sample dataset

Provider
aws | Google Cloud | Azure

Region
Singapore (ap-southeast-1)

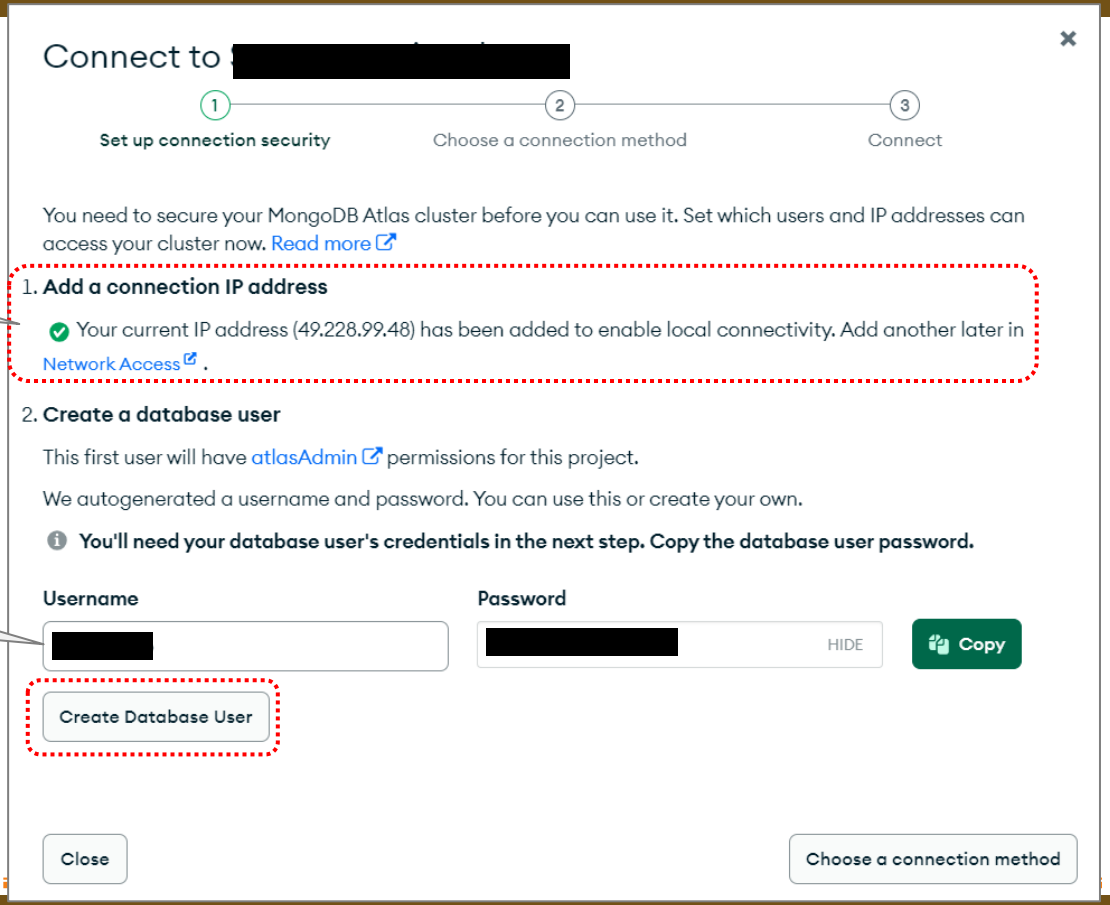
I'll do this later | Go to Advanced Configuration | Create Deployment

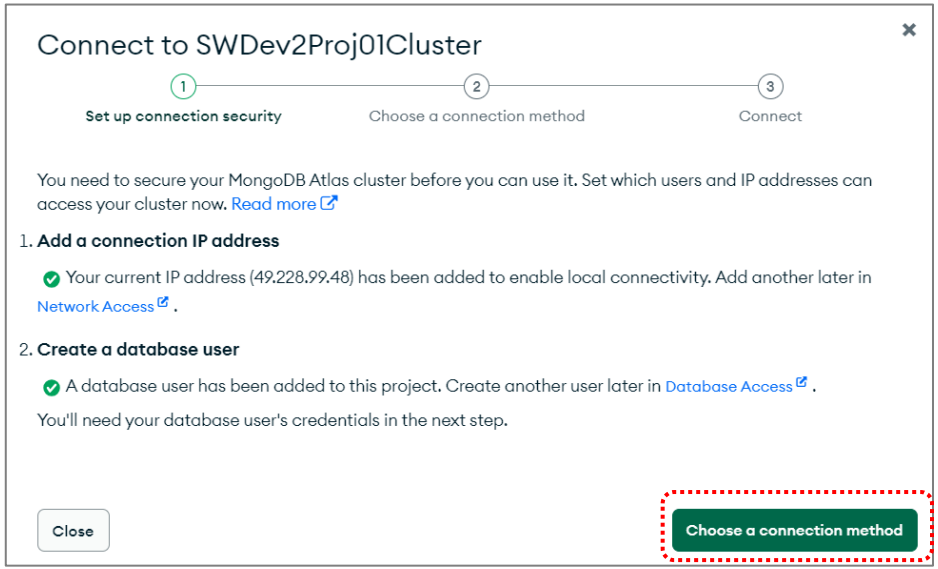
MongoDB Atlas Setup for your project

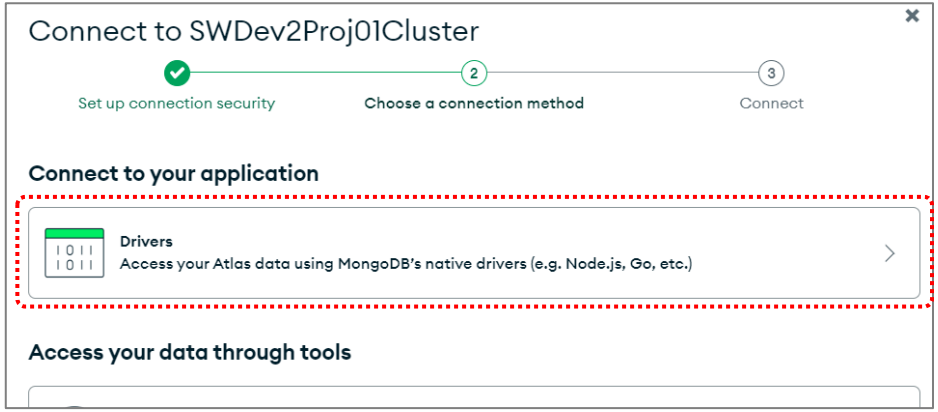
- **Configure Connection**

Add current IP address to allow connection

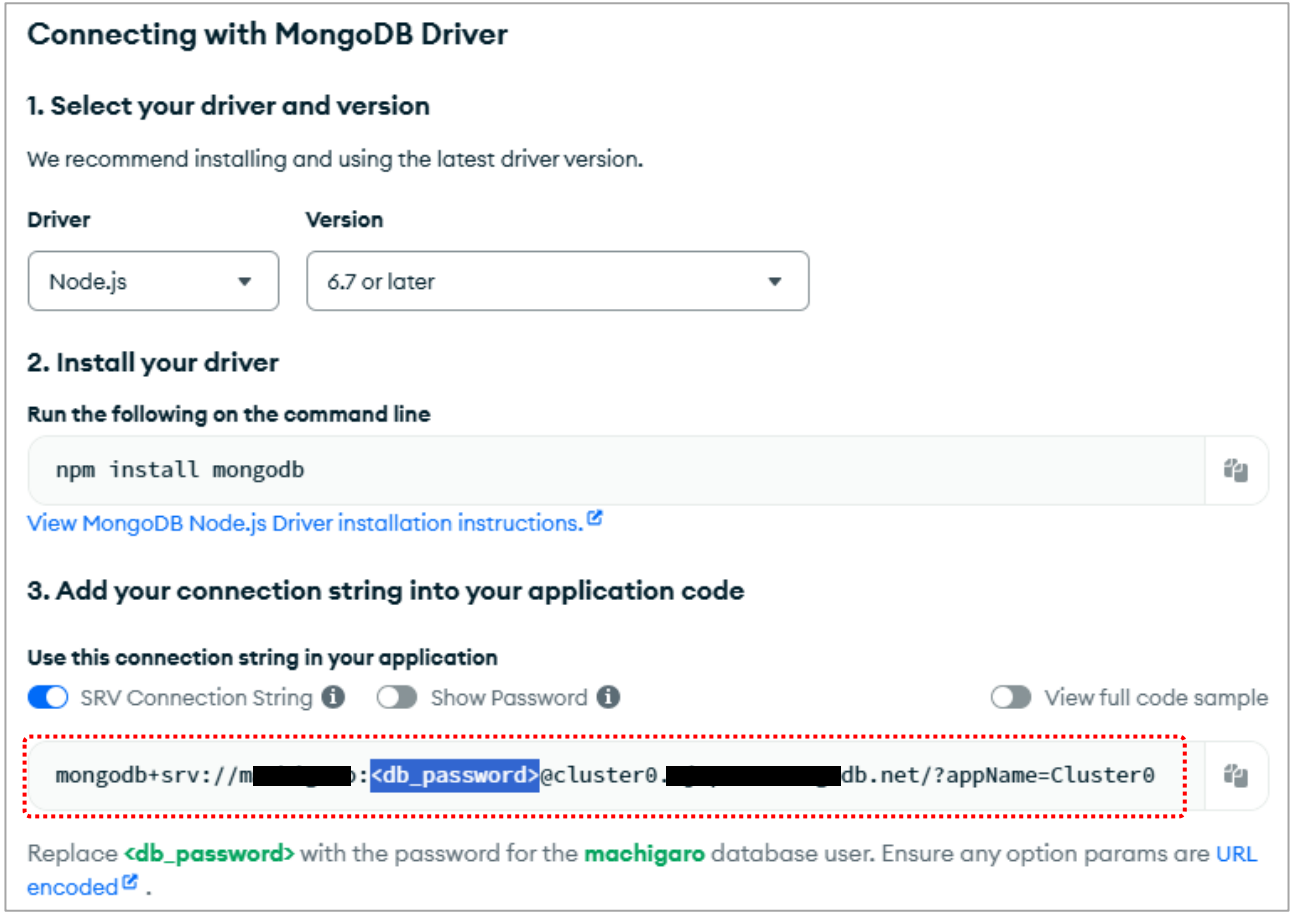
Create database user







- Setup Connection Method
  - Choose Connect to your application with "Drivers"

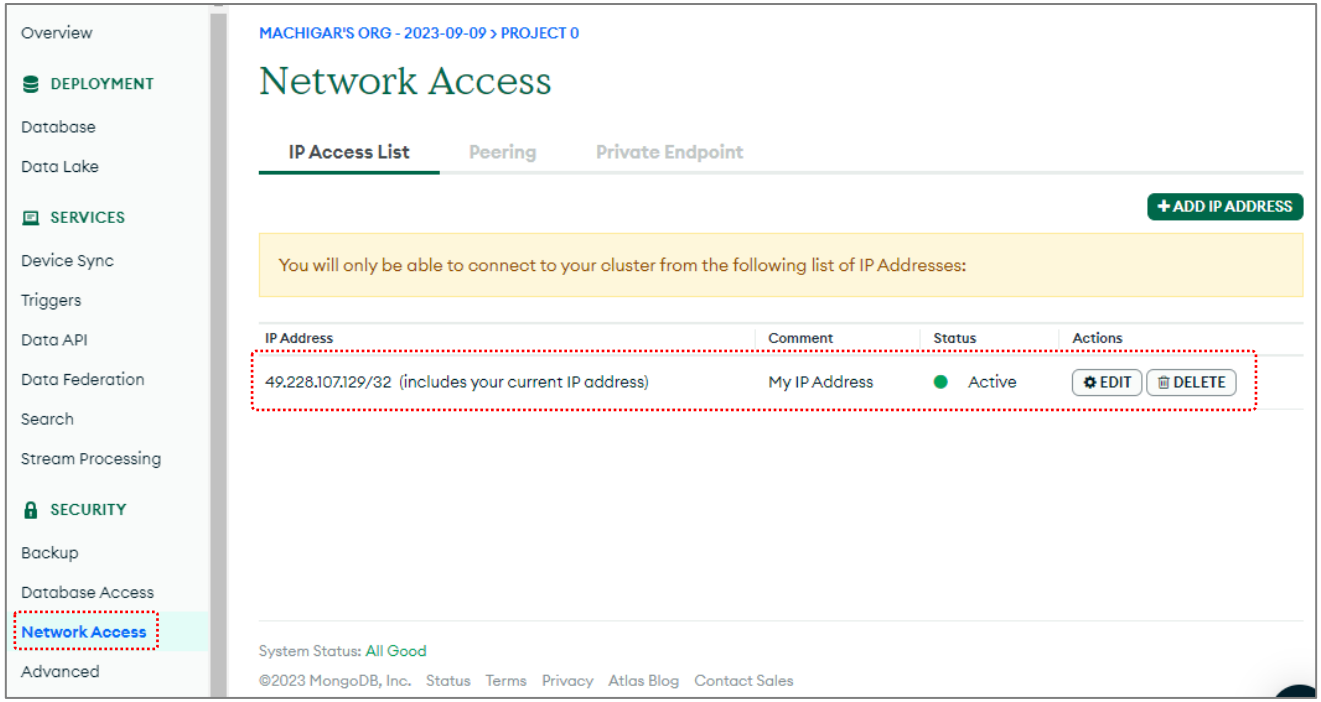


- Setup Connection Method
  - Copy connection string

MongoDB Atlas Setup for your project



- การเปลี่ยน allowed IP address ไปที่ project console เลือก “Network Access” Menu
  - Edit IP Address



Setup Backend API



- Clone Backend จาก GitHub Repository
  - `git clone https://github.com/SWDevPrac2-69-1/car-rental-backend.git`

- เปิดแล: backend project directory
  - Install mongoose (Object Data Modeling (ODM) library สำหรับ Node.js แล: MongoDB)

    ```
    npm install
    npm i mongoose
    ```

MongoDB Atlas Setup for your project



- ใน backend project directory, ไปที่ `config\config.env`
  - Declare `MONGO_URI` variable เป็น connection string

```
1  PORT=5000
2  NODE_ENV=development
3  MONGO_URI=mongodb+srv://[redacted]@cluster0.ojnpxw5.mongodb.net/?appName=Cluster0
4  JWT_SECRET=+AB44KcrVw8mTutUuI46CiBH/iCwg/RsvBHjH5jKLU4yVHbq8Q8+8qAIQySudDklPoJprX8ZosRuSbXJ3Tp44o6/H9D2mYJdkk3
   +8esAOtOH4T2/pslV/
   4zWVI1SLtrK66haCL1TUsazqUOvZFb6vE6gM2DlxPSbEE4lOQQsIi9ngC0qVuoQ985VmScNBuLnUCJCWYjJFg0MhRwT1yucvVaXeY0BKNNGAqa6Q
   ==
5  JWT_EXPIRE=365d
6  JWT_COOKIE_EXPIRE=365
7  AUTH_EMAIL=test@gmail.com
```

  - แทนที่ `<username>` ด้วย MongoDB Atlas username
  - แทนที่ `<password>` ด้วย MongoDB Atlas password

MongoDB Atlas Setup for your project



- Run backend application ใน backend project directory ด้วยคำสั่ง

  `npm run dev`

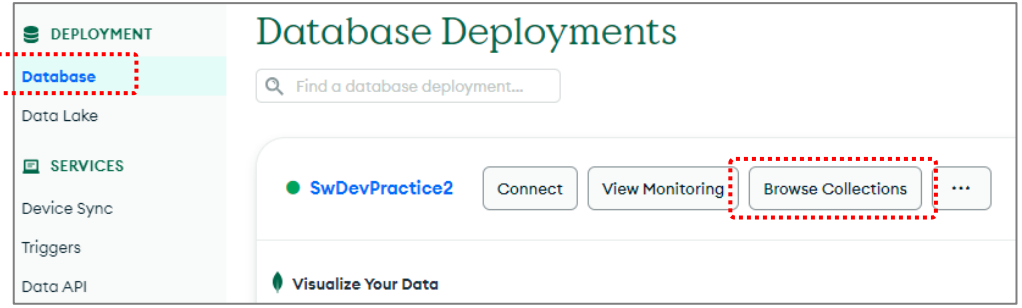
  - Backend application run ที่ port 5000

```
[nodemon] starting `node server.js`
Server running in development on http://localhost:5000
MongoDB Connected: swdev2proj01cluster-shard-00-02.otpib.mongodb.net
```

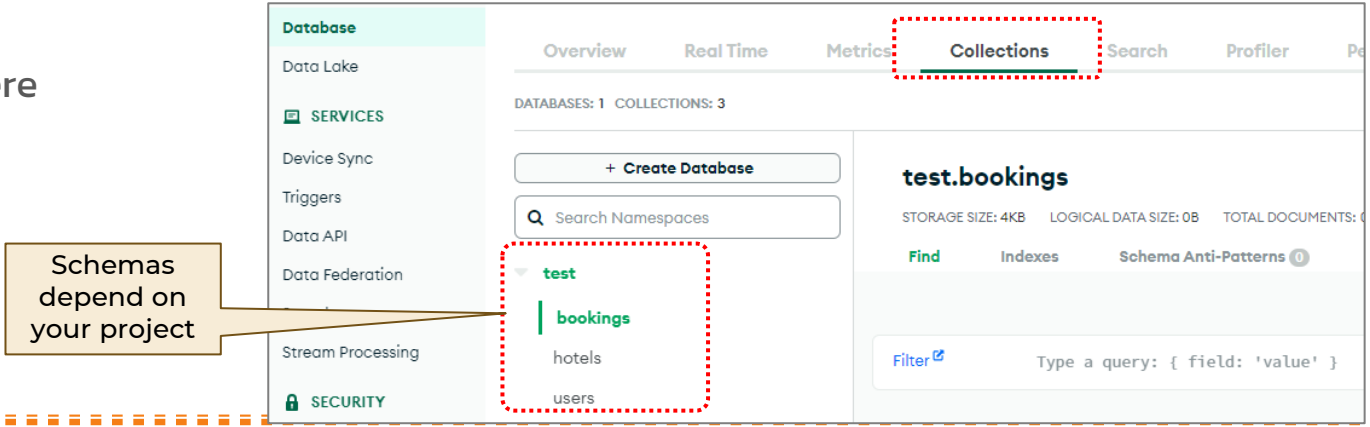
  - หยุดการทำงาน กด `ctrl + c`

MongoDB Atlas Setup for your project

- Check data collections on MongoDB Atlas
  - Goto Database > Browse Collections

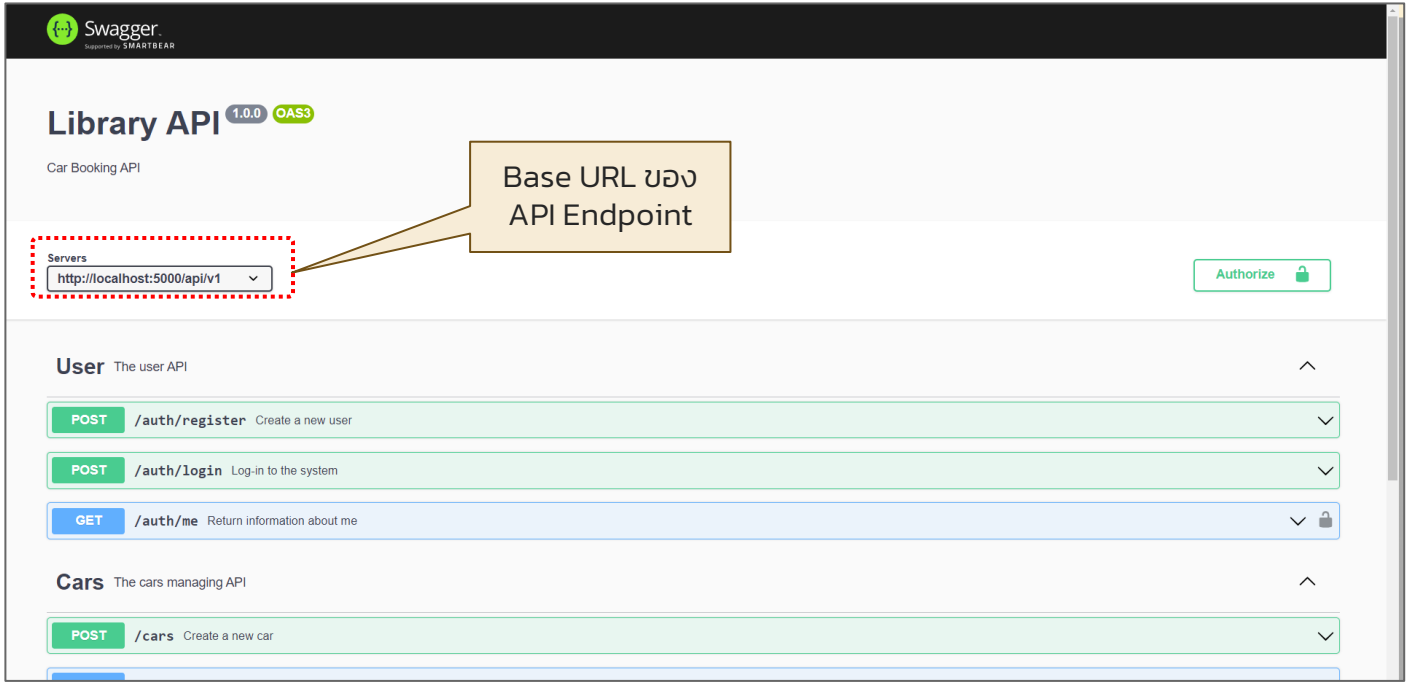


  - View the schema here



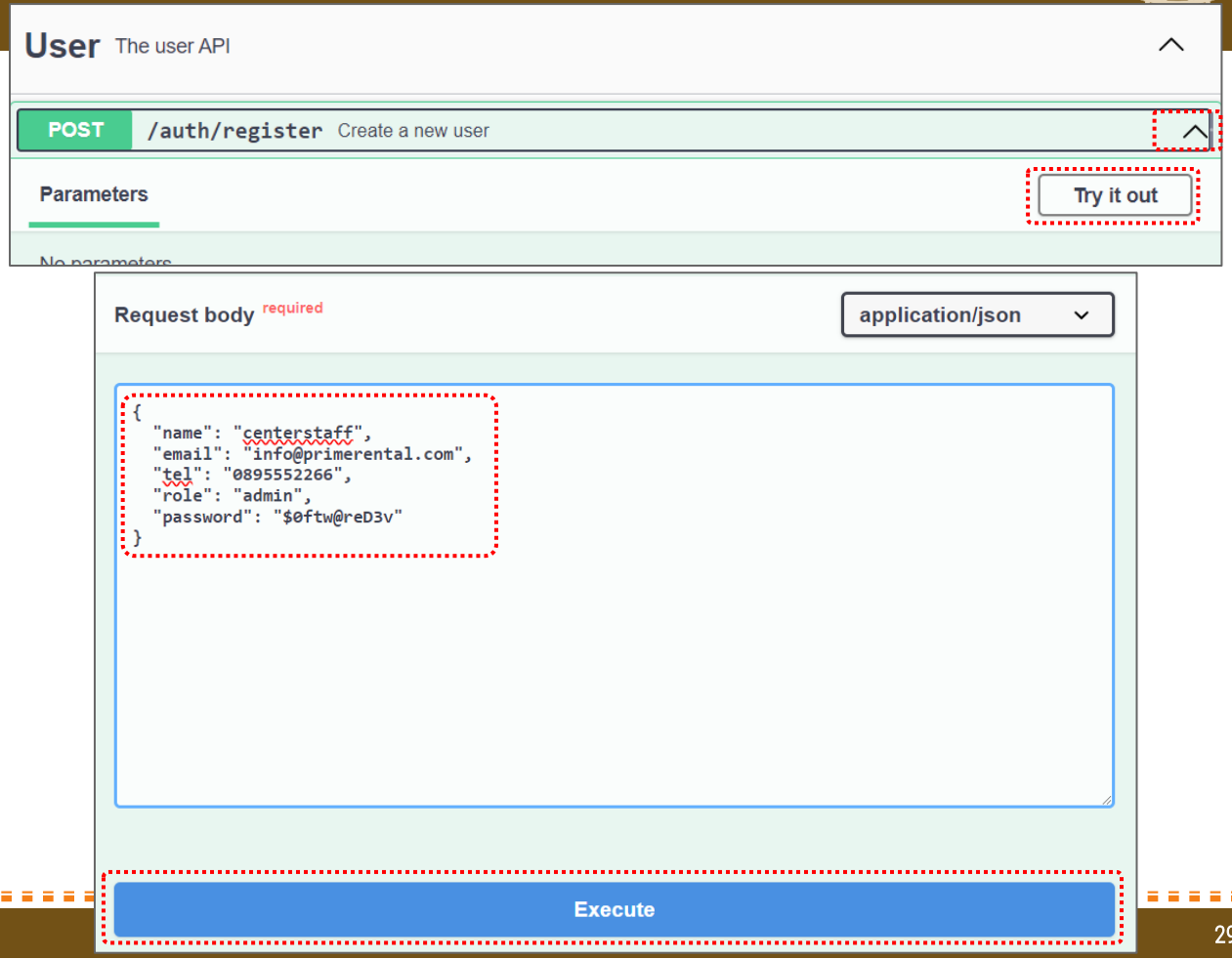
Exploring API

- ไปที่ API Document
  - http://localhost:5000/api-docs/



Exploring API

- ลองสร้าง Admin user



Exploring API

- ได้ response จาก success จากการสร้าง admin user
  - Copy Token ไว้ใส่ใน Authorization Header

Server response

Code | Details

200 *Undocumented*

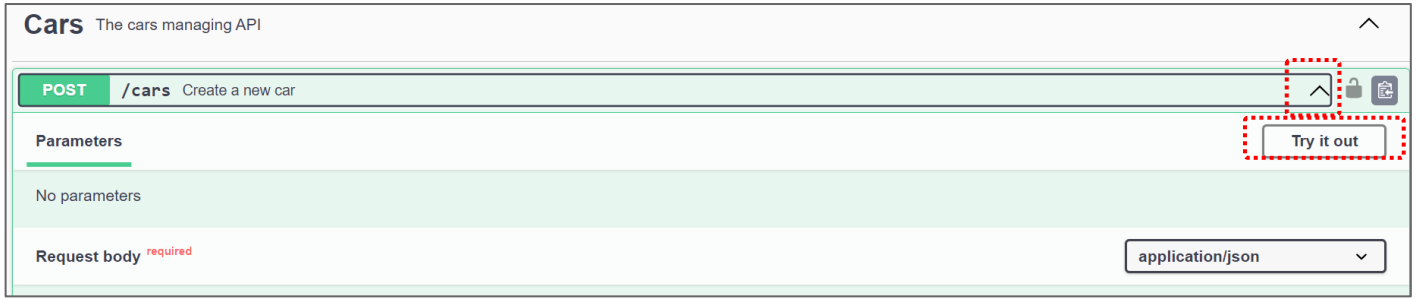
Response body


```
{
  "success": true,
  "_id": "651842b96dc6f322bf3a8db4",
  "name": "centerstaff",
  "email": "info@primerental.com",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY1MTg0MmI5NmRjNmYzMjJiZjNhOGRiNCIsImlhdCI6MTY5NjA4ODc2MSwiZXhwIjoxNzI3NjI0NzYxfQ.TrT8IHNiuAufPc39eWkwRa--8ATqG2Z7k1oMYrbQdYU"
}
```


Download

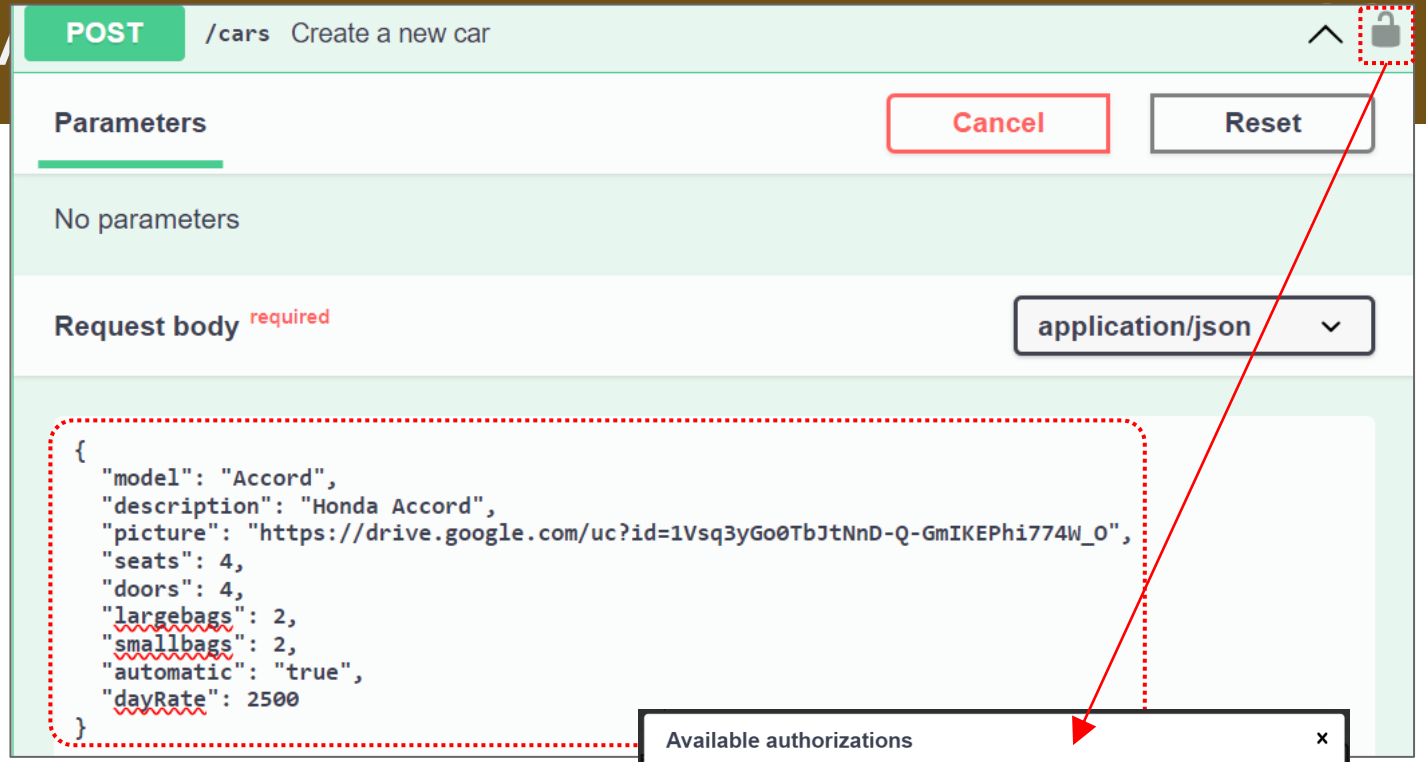
Response headers

- สร้างข้อมูล Car



Exploring A

- สร้างข้อมูล Car
  - ใส่ข้อมูล



- สร้างข้อมูล Car
  - ใส่ Authorization Header

Exploring API

- สร้างข้อมูล Car
  - สังเกต Server Response

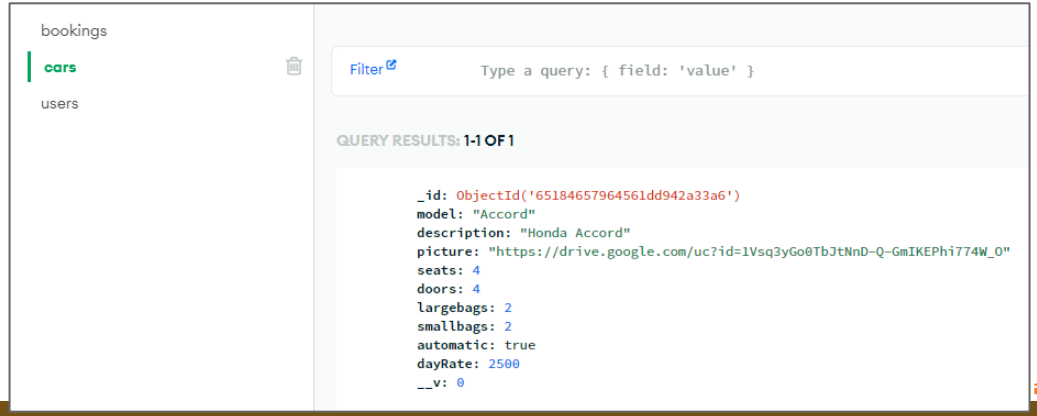
Server response

Code	Details
201	Response body


```
{
  "success": true,
  "data": {
    "model": "Accord",
    "description": "Honda Accord",
    "picture": "https://drive.google.com/uc?id=1Vsq3yGo0TbJtNnD-Q-GmIKEPhi774W_O",
    "seats": 4,
    "doors": 4,
    "largebags": 2,
    "smallbags": 2,
    "automatic": true,
    "dayRate": 2500,
    "_id": "65184657964561dd942a33a6",
    "__v": 0,
    "id": "65184657964561dd942a33a6"
  }
}
```

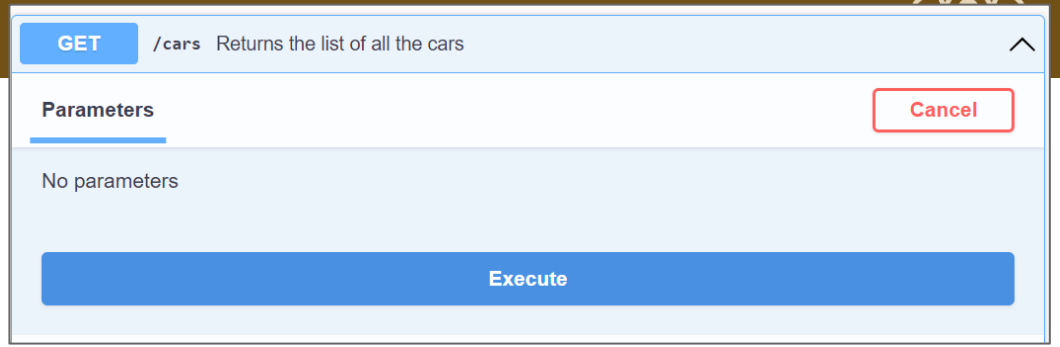

Download

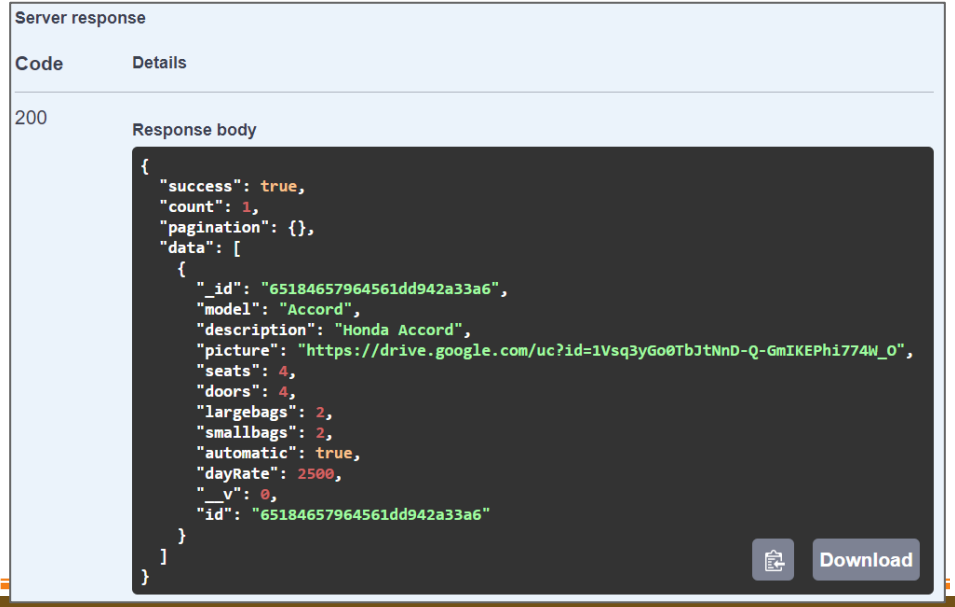
  - ดูข้อมูลใน database



Exploring API

- ใช้งาน API เพื่อดึงข้อมูล Car ทั้งหมด
  - สังเกต Server Response





Exploring API



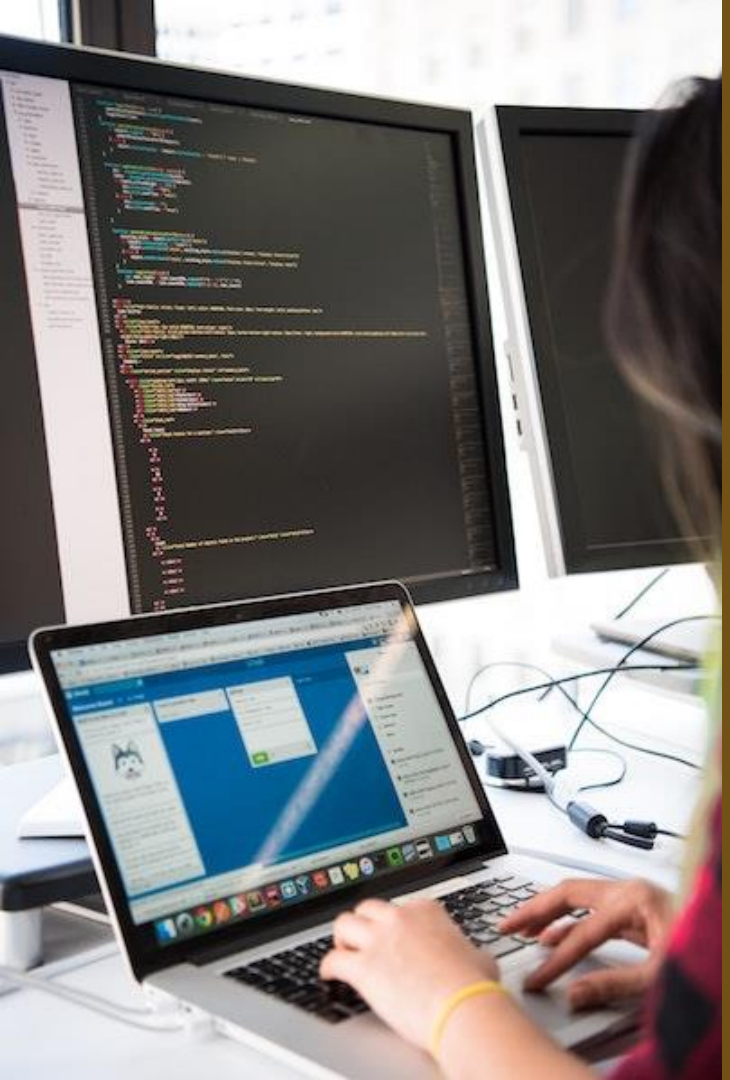
- ใส่ข้อมูล Car ลงไปรวม 4 โมเดล

model	description	picture	seats	doors	large bags	small bags	automatic	dayRate
Accord	Honda Accord	https://drive.google.com/uc?id=1Vsq3yGo0TbJtNnD-Q-GmIKEPhi774W_O	4	4	2	2	true	2500
Civic	Honda Civic	https://drive.google.com/uc?id=1qZjh3CmMFGEik3lcPFZZCif58q4cqSFe	4	4	2	1	true	1800
Fortuner	Toyota Fortuner	https://drive.google.com/uc?id=1_eyYyo6RfAg8zQKWji6FPXKcrnMQpars	5	5	3	3	true	2800
Model 3	Tesla Model 3	https://drive.google.com/uc?id=1glL1RmdkdVSUDOmqxQvz6L3_5f2bgPpj	4	4	2	1	true	2000

Exploring API

- ทดลองเรียก Public API ใน Web Browser
  - http://localhost:5000/api/v1/cars



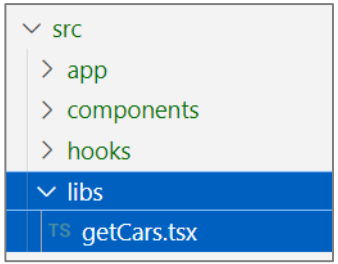


Workshop 9.2:
Server-Side Data Fetching

Server-Side Data Fetching



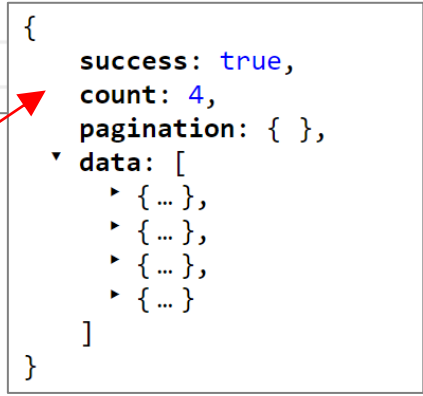
- สร้างโฟล์เดอร์ `libs` ใน `src` ไว้เก็บ function การทำงานต่าง ๆ
- สร้าง `libs\getCars.tsx` เก็บฟังก์ชั่นสำหรับ fetch ข้อมูล cars ทั้งหมด



src > libs > getCars.tsx > getCars

```tsx
export default async function getCars() {
    const response = await fetch("http://localhost:5000/api/v1/cars")
    if(!response.ok) {
        throw new Error("Failed to fetch cars")
    }
    return await response.json()
}
```

Response body จะเป็นรูปแบบของ `JSON Object` ซึ่งภายในบรรจุ attributes ต่าง ๆ เช่น `count`, `pagination`, รวมถึง `data` ที่แบบ array ของ `JSON Object` บรรจุข้อมูล cars



Server-Side Data Fetching

- สร้าง Server Component นำเสนอ Car Catalog โดยแสดงรายการรถผ่าน `ProductCard` เหมือน `CarPanel`
  - `ProductCard` ที่จะใช้ ไม่ต้องการปุ่ม `compare`



- สร้าง `components\CarCatalog.tsx` สำหรับนำเสนอ Car Catalog
  - รับ prop เป็น JSON Object ที่ได้จากการ parse response จากการดึงรายการ Car จาก API

```tsx
export default function CarCatalog({carJson}:{carJson:Object}) {
    return (
        <>
        Explore {carJson.count} models in our catalog

        </>
    )
}
```

เราไม่ได้นิยามคลาสสำหรับ car object ที่มีโครงสร้างแบบ `JSON Object` ที่ได้มา แต่กำหนด data type เป็น Object จึงทำให้ไม่รู้จัก JSON Attribute


```
{
    success: true,
    count: 4,
    pagination: { },
  ▾ data: [
      ▸ { … },
      ▸ { … },
      ▸ { … },
      ▸ { … }
    ]
}
```

Server-Side Data Fetching



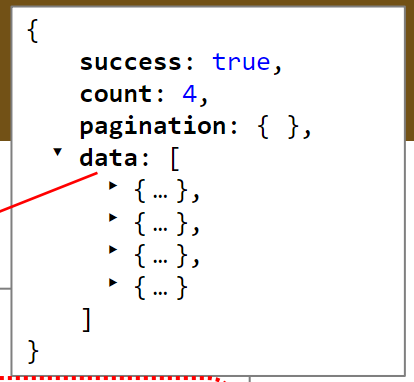
- ปรับ components\ProductCard.tsx
  - ให้ props onCompare เป็น optional
  - แสดงปุ่ม compare เมื่อมีการส่ง prop onCompare มาให้ ProductCard

```
export default function ProductCard ( { carName, imgSrc, onCompare }
    : { carName:string, imgSrc:string, onCompare?:Function }) {

    return (
        <InteractiveCard contentName={ carName }>
            <div className='w-full h-[70%] relative rounded-t-lg'>
            <Image src={imgSrc}
                alt='Product Picture'
                fill={true}
                className='object-cover rounded-t-lg'/>
            </div>
            <div className='w-full h-[15%] p-[10px]'>{carName}</div>
            {
                onCompare? <button className='block h-[10%] text-sm rounded-md bg-sky-600
                hover:bg-indigo-600 mx-2 px-1 py-1 text-white shadow-sm'
                onClick={ (e)=>{ e.preventDefault(); onCompare(carName) } }
                >Compare</button>:''
            }
        </InteractiveCard>
    );
}
```

Server-Side Data Fetching

- ใน components\CarCatalog.tsx
  - นำข้อมูลแต่ละ car มาแสดงลง ProductCard




```
<>
Explore {carJson.count} models in our catalog
<div style={{margin:"20px", display:"flex",
    flexDirection:"row", alignContent:"space-around",
    justifyContent:"space-around", flexWrap:"wrap", padding:"10px"}}>
        {
            carJson.data.map((carItem:Object)=>(
                <Link href={`/car/${carItem.id}`} key={carItem.id} className="w-1/5">
                <ProductCard carName={carItem.model} imgSrc={carItem.picture}/>
               </Link>
            ))
        }
</div>
</>
```

Server-Side Data Fetching



- ใน `(carinfo)\car\page.tsx` ซึ่งเป็นหน้าแสดงรายการ car ทั้งหมด
  - เปลี่ยน functional component เป็น `async` function และเรียก `getCars()` ด้วย `await` เพื่อ fetch data นำ JSON response ที่ได้ส่งเป็น prop ให้ `CarCatalog`

```
1  import getCars from "@/libs/getCars"
2  import CarCatalog from "@/components/CarCatalog"
3
4  export default async function Car() {
5      const cars = await getCars()
6      return (
7          <main className="text-center p-5">
8              <h1 className='text-xl font-medium'>Select Your Travel Partner</h1>
9              <CarCatalog carJson={cars}/>
10         </main>
11     )
12 }
```

Unhandled Runtime Error

Error: Invalid src prop (https://drive.google.com/uc?id=1Vsq3yGo0TbJtNnD-Q-GmIKEPhi774W_O) on `next/image`, hostname "drive.google.com" is not configured under images in your `next.config.js`
See more info: https://nextjs.org/docs/messages/next-image-unconfigured-host

  - ไปที่ http://localhost:3000/car จะพบว่ามี Error จากการดึง image เพราะยังไม่ได้ configure domain ของ image

Server-Side Data Fetching

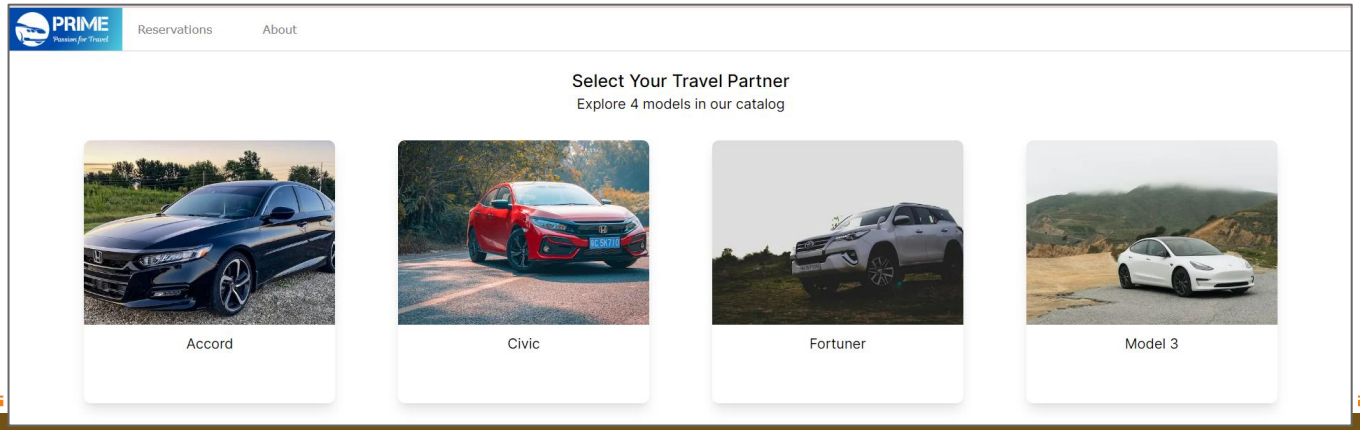
```ts
const nextConfig: NextConfig = {
  images: {
      domains: ['drive.google.com'],
  }
};

export default nextConfig;
```

- แก้ไข `next.config.ts` เพิ่ม
  - Domain สำหรับ image ให้รองรับการดึง image จาก drive.google.com

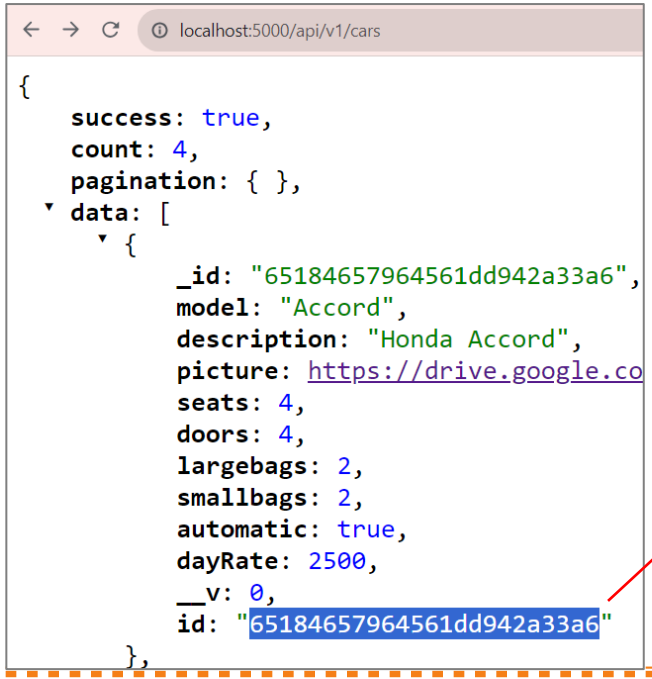
```
- warn
> Found a change in next.config.js. Restarting the server to apply the changes...
PS D:\SoftwareDevPrac2\ReactWeb\instructor_wk09\car-rental-app> npm run dev
```

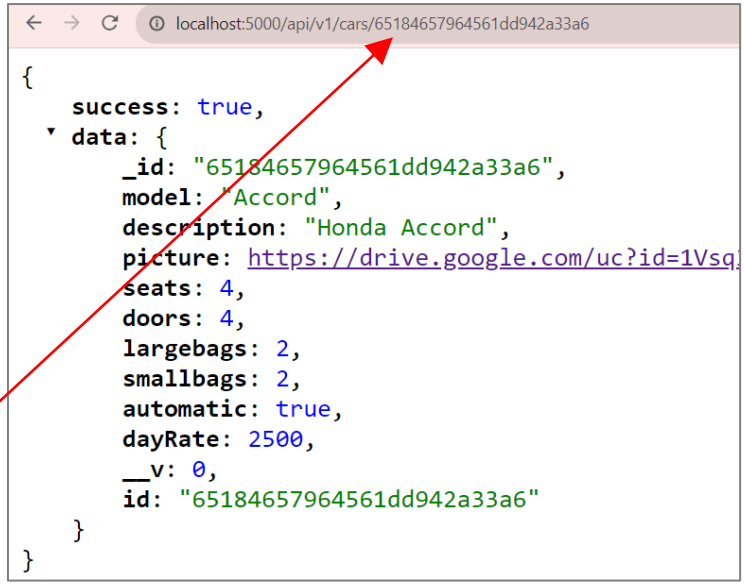
- Restart Next.js server
- ไปที่ http://localhost:3000/car
- จะได้ภาพรถแสดงบน `CarCatalog`



Server-Side Data Fetching

- สำรวจ Public API สำหรับดึงข้อมูล Car แต่ละ id
  - จาก http://localhost:500/api/v1/cars
  - นำ id ของ Car มาใช้เรียก http://localhost:5000/api/v1/cars/{id}

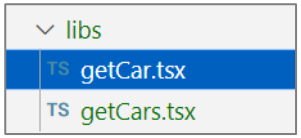




Server-Side Data Fetching



- สร้าง `libs\getCar.tsx` เก็บฟังก์ชั่นสำหรับ fetch ข้อมูล car แต่ละโมเดล



libs > getCar.tsx > getCar

```tsx
export default async function getCar(id:string) {
    const response = await fetch(`http://localhost:5000/api/v1/cars/${id}`)
    if(!response.ok) {
        throw new Error("Failed to fetch cars")
    }
    return await response.json()
}
```

- ใน `(carinfo)\car\[cid]\page.tsx` ซึ่งเป็นหน้าแสดงรายการ car แต่ละโมเดล
  - เรียก `getCar()` ด้วย `await`

```tsx
import Image from 'next/image';
import getCar from '@/libs/getCar';

export default async function CarDetailPage({params}: {params: Promise<{cid: string}>}) {

    const { cid } = await params
    const carDetail = await getCar(cid)
```

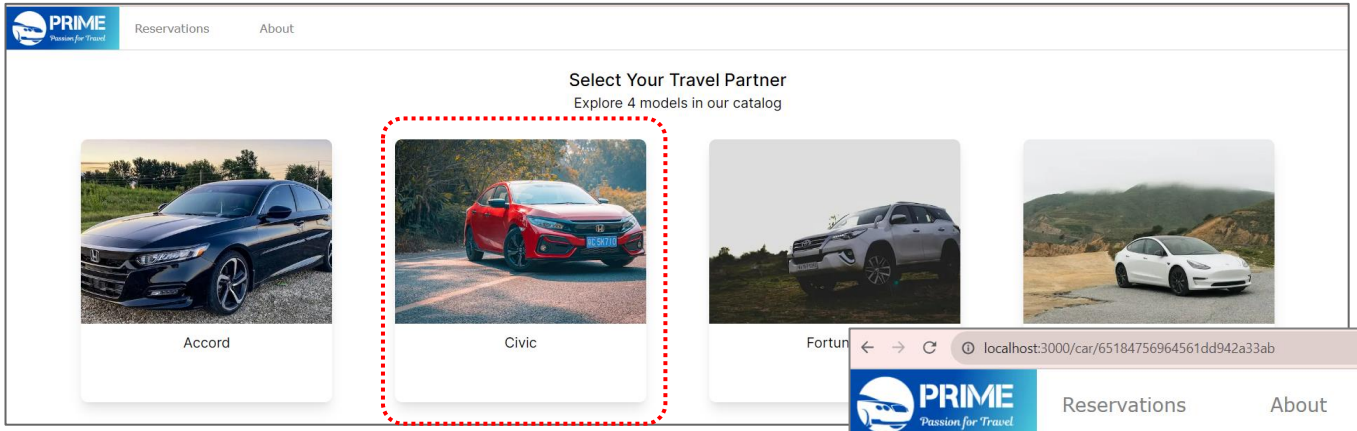
Server-Side Data Fetching

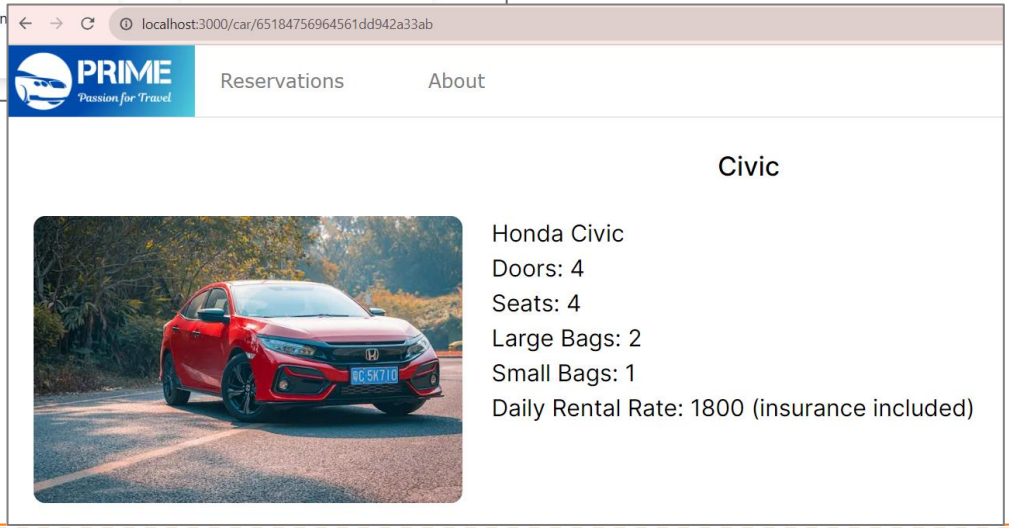


- ใน (carinfo)\car\[cid]\page.tsx
  - ยกเลิกการใช้ Mock Data
  - นำ carDetail ที่ได้มาแสดง

```
/*
const mockCarRepo = new Map()
mockCarRepo.set("001", {name:"Honda Civic", image:"/img/civic.jpg"})
mockCarRepo.set("002", {name:"Honda Accord", image:"/img/accord.jpg"})
mockCarRepo.set("003", {name:"Toyota Fortuner", image:"/img/fortuner.jpg"})
mockCarRepo.set("004", {name:"Tesla Model 3", image:"/img/tesla.jpg"})
*/
return (
    <main className="text-center p-5">
      <h1 className='text-lg font-medium'>{carDetail.data.model}</h1>
      <div className="flex flex-row my-5">
      <Image src={carDetail.data.picture}
          alt='Product Picture'
          width={0} height={0} sizes="100vw"
          className='rounded-lg w-[30%] bg-black'/>
      <div className='text-md mx-5 text-left'>{carDetail.data.description}
      <div>Doors: {carDetail.data.doors}</div>
      <div>Seats: {carDetail.data.seats}</div>
      <div>Large Bags: {carDetail.data.largebags}</div>
      <div>Small Bags: {carDetail.data.smallbags}</div>
      <div>Daily Rental Rate: {carDetail.data.dayRate} (insurance included)</div>
      </div>
      </div>
    </main>
)
```

Server-Side Data Fetching

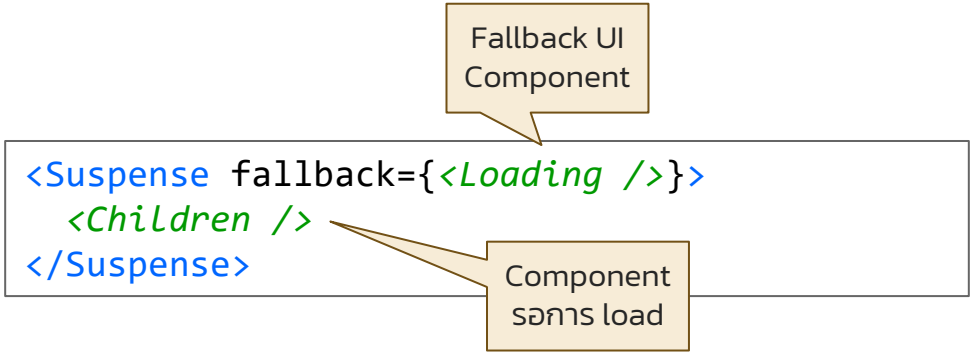


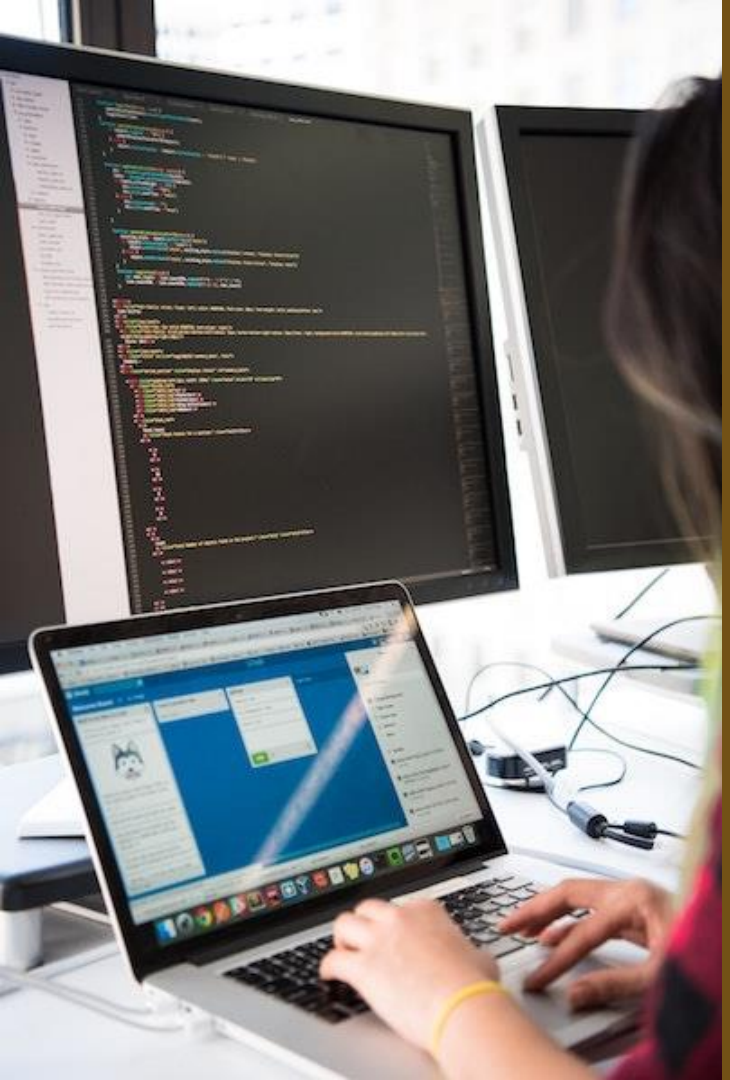


Suspense

- Suspense เป็น React feature ที่เอื้อให้ component ระงับ (suspend) การ render เพื่อรอการดำเนินการอย่างใดอย่างหนึ่ง
  - เหมาะกับการรอ Asynchronous operation เช่น การ load data
- React Suspense ทำงานโดยการโยน Promise ที่ได้รับมาจาก component เพื่อให้ React จัดการการ render สำหรับ component นั้น และแสดง fallback UI จนกว่า promise จะถูก resolve

- `<Suspense>` ใช้สำหรับแสดง fallback UI จนกว่า children ของ จะ load เสร็จ





Workshop 9.3: Render-as-you-fetch & Suspense

Loading Delay

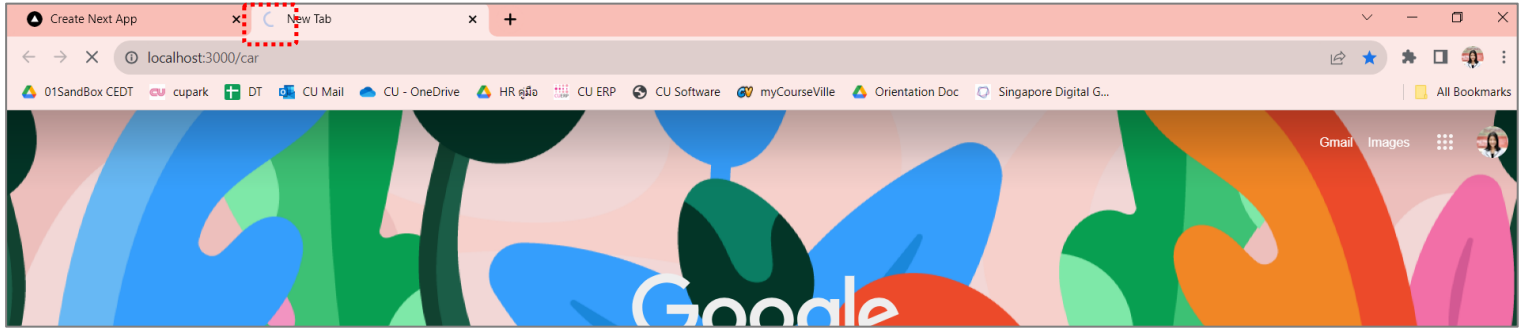
- ลองใช้งาน Loading UI ในกรณีที่ loading time มีระยะเวลานาน
- จำลองการ loading นาน ๆ ใน `getCars()` โดย
  - เพิ่ม time out 5 วินาที

```
export default async function getCars() {

  // add timeout for loading delay testing
  await new Promise((resolve)=>setTimeout(resolve, 5000))

  const response = await fetch("http://localhost:5000/api/v1/cars")
  if(!response.ok) {
    throw new Error("Failed to fetch cars")
```

  - ลอง load จะพบว่า หน้าจะ delay และรอให้ load สำเร็จถึงจะปรากฏต่อหน้าผู้ใช้



Render-as-you-fetch



- ใน (carinfo)\car\page.tsx ลองใช้งาน Suspense เพื่อให้ Component อื่นแสดงก่อน แล้วแสดงข้อความ Loading พร้อม Linear Progress Bar ระหว่างรอ CarCatalog load
  - ปรับ page (carinfo)\car\page.tsx ให้ไม่เป็น async component และนำ await ออก
  - นำ Suspense ไปครอบ CarCatalog component

```
import getCars from "@/libs/getCars"
import CarCatalog from "@/components/CarCatalog"
import {Suspense} from "react"
import { LinearProgress } from "@mui/material"

export default function Car() {
    const cars = getCars()
    return (
        <main className="text-center p-5">
            <h1 className='text-xl font-medium'>Select Your Travel Partner</h1>
            <Suspense fallback={<p>Loading...<LinearProgress/></p>}>
            <CarCatalog carJson={cars}/>
            </Suspense>
        </main>
    )
}
```

Render-as-you-fetch

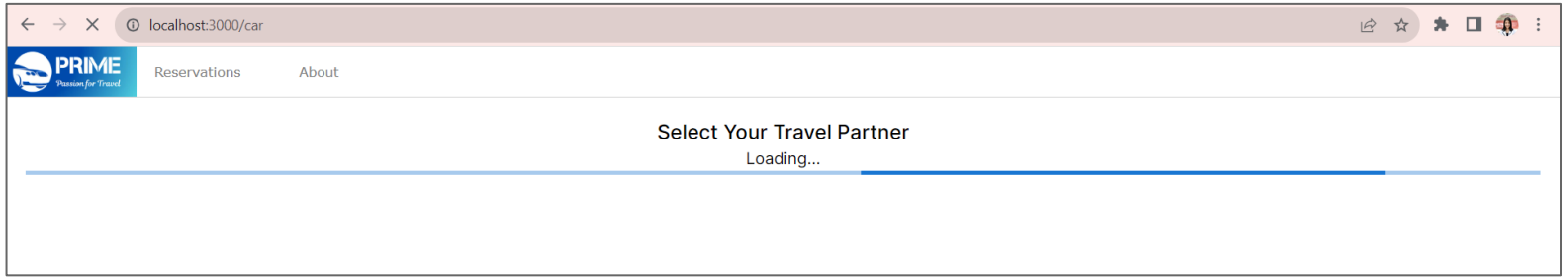


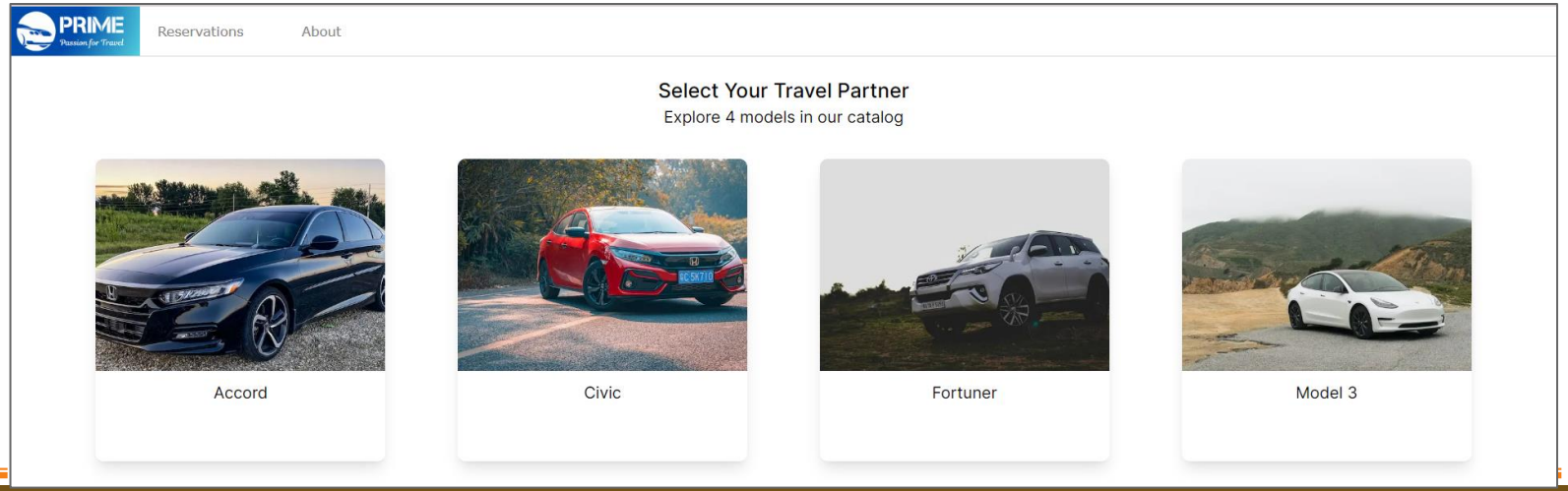
- ใน `components\CarCatalog.tsx` ปรับเป็น `async` functional component
  - ค่าที่ส่งเข้ามาเป็น prop `carJson` จะเป็น `promise` จากการ fetch ข้อมูล ให้ `await` รอค่า
  - นำค่าที่พร้อมใช้ไปใช้งานต่อ

```
export default async function CarCatalog({carJson}:{carJson:Object}) {
    const carJsonReady = await carJson
    return (
        <>
        Explore {carJsonReady.count} models in our catalog
        <div style={{margin:"20px", display:"flex",
            flexDirection:"row", alignContent:"space-around",
            justifyContent:"space-around", flexWrap:"wrap", padding:"10px"}}>
                {
                  carJsonReady.data.map((carItem:Object)=>(
                        <Link href={`/car/${carItem.id}`} className="w-1/5">
                        <ProductCard carName={carItem.model} imgSrc={carItem.picture}/>
                       </Link>
                    ))
                }
        </div>
        </>
```

Render-as-you-fetch

- ไปที่ http://localhost:3000/car



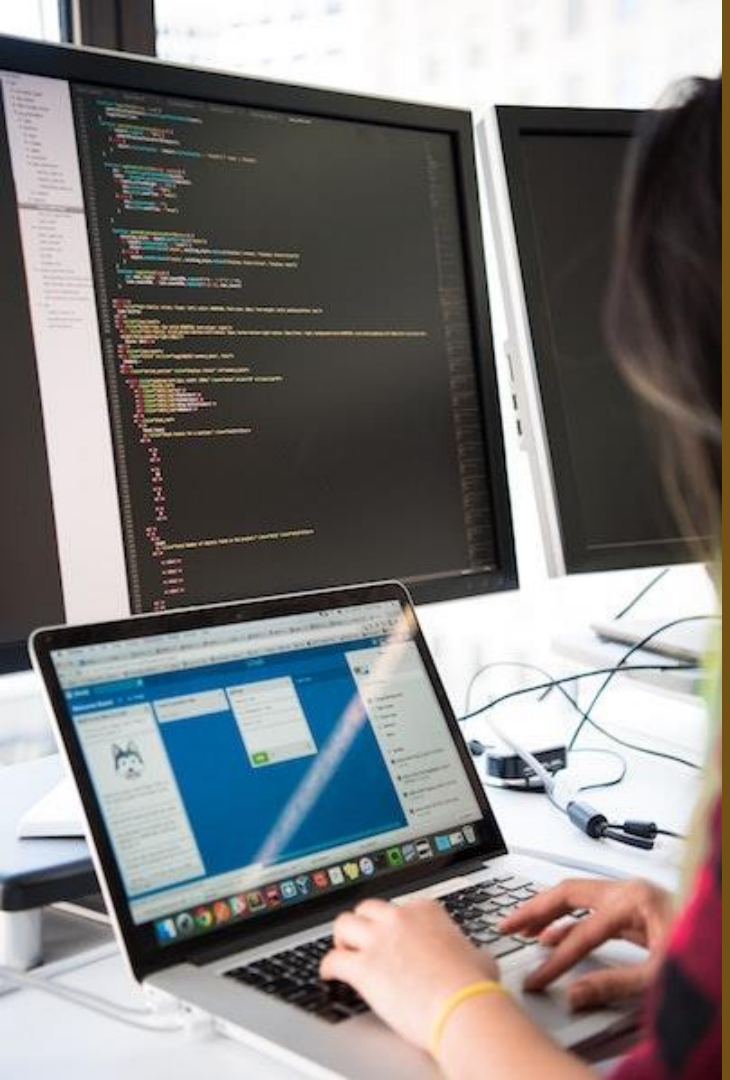


Client-Side Data Fetching



- การทำ Data Fetching บน Server-Side เป็นแนวทางที่แนะนำ อย่างไรก็ตาม การทำ data fetching บน Client-side ยังมีประโยชน์ ในหลายสถานการณ์
  - ไม่จำเป็นต้องมี SEO indexing หรือไม่จำเป็นต้อง pre-render ข้อมูล
  - มีการ update content เป็นประจำ หรือกระทำเมื่อมี interaction จากผู้ใช้

- Client-side data fetching ดำเนินการต่างจาก server-side rendering APIs โดยนิยมทำ client-side data fetching ในระดับ component
  - หาก fetch ที่ระดับ page เมื่อ data ถูก fetch ช่วง runtime แล้วจึง update content ของทั้ง page เมื่อ data เปลี่ยน
  - หาก fetch ที่ระดับ component เมื่อ data ถูก fetch ช่วง component mount แล้วจึง update content ของ component เมื่อ data เปลี่ยน
- การใช้งาน client-side data fetching อาจกระทบประสิทธิภาพการทำงาน รวมถึงความเร็วในการ load หน้า เนื่องจากทำเมื่อ component หรือ page ได้รับการ mount และ data ไม่ถูก cache



Workshop 9.4:
Client-Side Fetch-then-render

Client-Side Fetch-then-render



- ใน `components\CarPanel.tsx` ซึ่งเป็น client-side component ปรับให้มีการ fetch data
  - ใช้งาน `useEffect` เพื่อ fetch data ตอน component mount
  - จากนั้นเมื่อได้ data แล้ว ให้นำ data ไป update เป็นค่าให้กับ state variable `carResponse` ซึ่งจะทำให้เกิดการ re-render

```
import Link from "next/link"
import { useRef, useEffect } from "react"
import getCars from "@/libs/getCars"

export default function CarPanel() {

    const [carResponse, setCarResponse] = useState(null)

    useEffect(()=>{
        const fetchData = async () => {
            const cars = await getCars()
            setCarResponse(cars)
        }
        fetchData()
    },[])


    const countRef = useRef(0)
    const inputRef = useRef<HTMLInputElement>(null)
```

Client-Side Fetch-then-render



- ใน components \CarPanel.tsx

ถ้า carResponse ยังไม่ได้รับการตั้งค่าให้ return component แสดงสถานะว่า loading

ถ้า carResponse ได้รับการตั้งค่าแล้ว จะ return รายการ ProductCard บรรจุข้อมูล car

```
/*
const mockCarRepo = [{cid:"001", name:"Honda Civic", image:"/img/civic.jpg"},
        {cid:"002", name:"Honda Accord", image:"/img/accord.jpg"},
        {cid:"003", name:"Toyota Fortuner", image:"/img/fortuner.jpg"},
        {cid:"004", name:"Tesla Model 3", image:"/img/tesla.jpg"}]
*/
```

ยกเลิกการใช้งาน Mock Data

```
if(!carResponse) return (<p>Car Panel is Loading ...</p>)

return (
    <div>
        <div style={{margin:"20px", display:"flex",
        flexDirection:"row", alignContent:"space-around",
        justifyContent:"space-around", flexWrap:"wrap", padding:"10px"}}>
            {
              carResponse.data.map((carItem)=>(
                <Link href={`/car/${carItem.id}`} className="w-1/5" key={carItem.id}>
                <ProductCard carName={carItem.model} imgSrc={carItem.picture}
                onCompare={ (car:string)=>dispatchCompare({type:'add', carName:car}) }
                />
                </Link>
                )
              )
            }
```

Client-Side Fetch-then-render



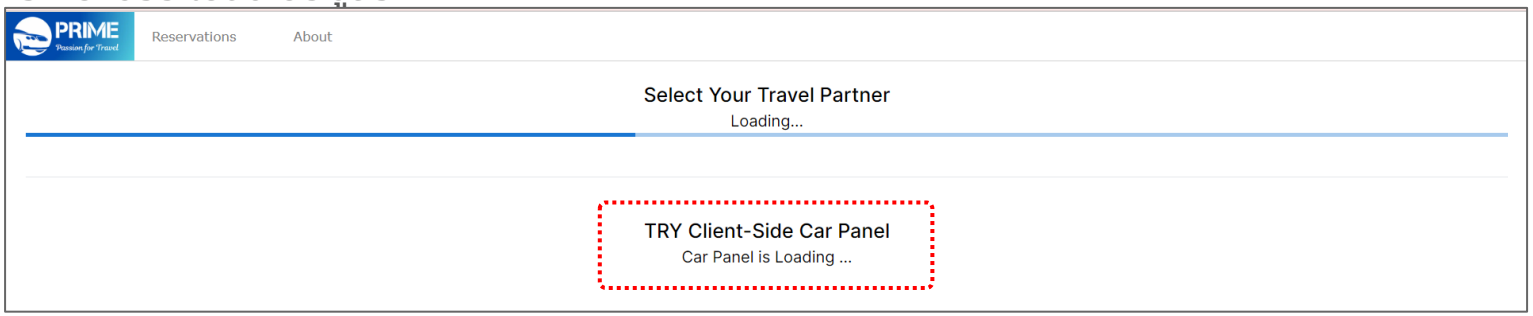
- ใน (carinfo)\car\page.tsx วางการใช้งาน CarPanel

```
import { LinearProgress } from "@mui/material"
import CarPanel from "@/components/CarPanel"

export default function Car() {
    const cars = getCars()
    return (
        <main className="text-center p-5">
            <h1 className='text-xl font-medium'>Select Your Travel Partner</h1>
            <Suspense fallback={<p>Loading...<LinearProgress/></p>}>
            <CarCatalog carJson={cars}/>
            </Suspense>
            <hr className="my-10"/>
            <h1 className='text-xl font-medium'>TRY Client-Side Car Panel</h1>
            <CarPanel/>
        </main>
    )
}
```

Client-Side Fetch-then-render

- ไปที่ http://localhost:3000/car
  - ระหว่างรอ load ข้อมูลรถ



  - เมื่อได้รับข้อมูล

